Software Architecture and Design Specification

Project: Chat Application Authors: Team 5 Date: 19-10-2026 Version: 1.0

Document Control

Version	Date	Author	Change Summary
1.0	19-10-2026	Team 5	Initial Draft - Architecture and UML finalized

1. Introduction

1.1 Purpose

This document specifies the architecture, module design, and network flows of the Chat Application system. It serves as the primary blueprint for backend and frontend implementation.

1.2 Scope

The system scope encompasses real-time user-to-user messaging, chat room management, online presence tracking, user authentication, and persistent message storage.

1.3 Audience

Developers, Quality Assurance (QA) engineers, system architects, and course instructors evaluating the Software Engineering mini-project.

1.4 Definitions

WSS: WebSocket Secure, used for real-time bi-directional communication.

JWT: JSON Web Token, utilized for stateless user authentication.

UI: User Interface.

API: Application Programming Interface.

2. Document Overview

2.1 How to use this document

This specification provides deliverables including UML component diagrams, sequence flows for core features, API structure, and the chosen technological architecture.

2.2 Related Documents

Software Requirements Specification (SRS), Software Test Plan (STP), and Requirements Traceability Matrix (RTM).

3. Architecture

3.1 Goals & Constraints

Goals: Real-time message delivery with low latency (<200ms), reliable message ordering, and high availability.
Constraints: Must operate securely over standard web ports (443), maintain lightweight client bundles, and ensure data integrity within a relational or document database.

3.2 Stakeholders & Concerns

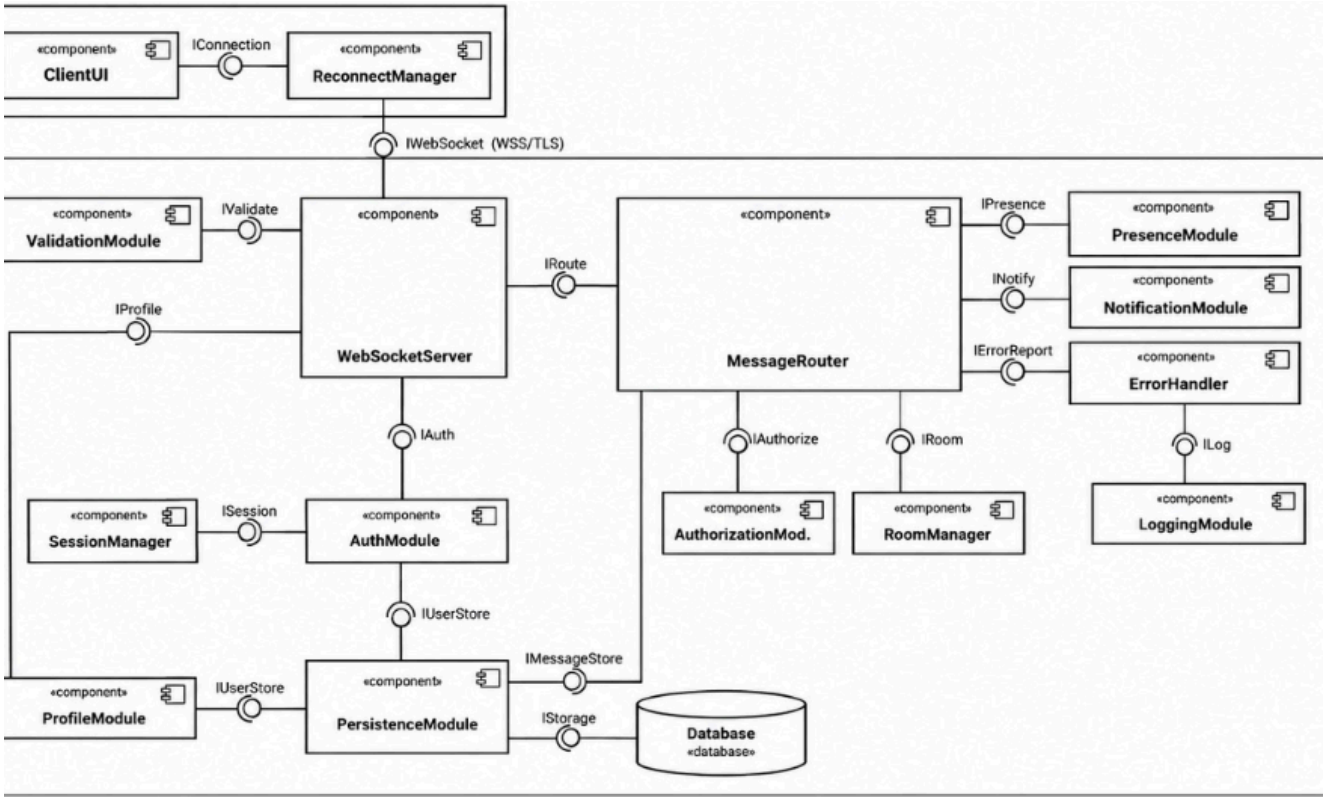
Users: Expect fast message delivery, privacy, and an intuitive interface.

System Administrators: Require stable server uptime and accessible monitoring logs.

Developers: Need a modular, decoupled architecture to divide workload efficiently.

3.3 Component (UML) Diagram

The following UML Component Diagram illustrates the structural relationships between the Client UI, the WebSocket server, message routing mechanisms, and data persistence modules.



3.4 Component Descriptions

ClientUI & ReconnectManager: Handles user input, displays chat histories, and automatically attempts to restore dropped connections.

WebSocketServer: The primary gateway handling incoming WSS connections and delegating packets.

AuthModule & SessionManager: Validates user credentials, issues JWTs, and tracks active sessions.

MessageRouter & RoomManager: Determines where a message belongs (direct user or specific room) and broadcasts it to the correct connected clients.

PresenceModule: Tracks and broadcasts user "online" or "offline" statuses.

PersistenceModule & Database: Stores user profiles and archives message history for retrieval.

3.5 Chosen Architecture Pattern and Rationale

A Client-Server Layered Architecture coupled with an Event-Driven pattern was chosen. WebSockets (event-driven) are utilized for real-time messaging, while standard REST APIs (layered) handle static requests like user registration and login. A full Microservices approach was rejected as overly complex for the scope of this mini-project.

3.6 Technology Stack & Data Stores

Frontend: React/HTML5. Backend: Node.js with WebSockets (or Python with Flask-SocketIO). Database: MySQL/PostgreSQL for structured persistence of user data and messages. Transport: TLS encryption (HTTPS/WSS).

3.7 Risks & Mitigations

Risk: WebSocket connection drops resulting in lost messages.
Mitigation: Implementation of a `ReconnectManager` on the client-side and message acknowledgment receipts on the server-side.

3.8 Traceability to Requirements

Req-Auth: Traced to AuthModule and ValidationModule.

Req-Chat: Traced to WebSocketServer, MessageRouter, and ClientUI.

3.9 Security Architecture

Threat Modeling (STRIDE):
Spoofing prevented via JWT validation. Tampering mitigated by WSS encryption. Information Disclosure prevented by sanitizing logs. Denial of Service (DoS) handled by connection rate limiting per IP.

4. Design

4.1 Design Overview

The Chat Application is designed with modular backend components, isolating authentication, active session routing, and database persistence to allow concurrent development by team members.

4.2 UML Sequence Diagrams

The tables below describe the step-by-step interactions for two primary system flows.

Flow 1: User Login and WebSocket Handshake

Step	Component	Action / Message
1	ClientUI	User submits credentials (username/password) via HTTPS POST.
2	AuthModule	Validates credentials against Database. Generates JWT.
3	ClientUI	Receives JWT. Initiates WebSocket upgrade request using JWT.

4	SessionManager	Validates JWT on socket connection. Marks user session as active.
5	PresenceModule	Broadcasts "User Online" event to authorized contacts.

Flow 2: Sending a Chat Message

Step	Component	Action / Message
1	ClientUI	User types message and emits sendMessage event over WSS.
2	WebSocketServer	Receives payload and passes to ValidationModule (checks empty strings/length).
3	PersistenceModule	Saves the message to the Database asynchronously with timestamp.
4	MessageRouter	Identifies recipient(s) active socket connections.
5	WebSocketServer	Emits receiveMessage event to target ClientUI(s).

4.3 API Design

Below are interface definitions for two core backend boundaries.

Endpoint: /api/auth/login Method: POST Description: Authenticates a user and returns a secure token. Request Body: { "username": "user1", "password": "securepassword" } Response (Success): { "status": "ok", "token": "eyJhbGciOi...", "userId": "101" } Errors: 401 Unauthorized (invalid credentials), 400 Bad Request

WebSocket Event: sendMessage Direction: Client -> Server Payload: { "roomId": "room-5", "text": "Hello team!", "timestamp": 1694250000 } Server Action: Validates payload, stores in DB, routes to RoomManager. Server Response (to room): emit('newMessage', { "userId": "101", "text": "Hello team!" })

4.4 Error Handling, Logging & Monitoring

Error Handling: Standardized JSON error structures are used across APIs. WebSocket drops trigger exponential backoff reconnections.
Logging: The LoggingModule records server errors, auth failures, and routing faults. No sensitive user messages or passwords will be written to logs.
Monitoring: System health is monitored via active WebSocket connection counts, message delivery failure rates, and database query latency.

4.5 UX Design

The Chat UI is designed to be responsive, featuring a sidebar for active rooms/contacts and a main reading pane. It includes visual typing indicators and clear error toasts for network disconnections.

4.6 Open Issues & Next Steps

Future enhancements beyond the mini-project scope may include end-to-end encryption (E2EE) for messages, push notifications for offline users, and multimedia file sharing support.

5. Appendices

5.1 Glossary: JWT, WSS, TLS, API, RoomManager.
5.2 References: IEEE Std 1471-2000, OWASP Top 10 Security Risks.
5.3 Tools: Node.js, Socket.IO/Flask-SocketIO, Draw.io (UML design), Postman (API testing). Chat